3. Line justification.

3.1. Locate word bounds (using ICU break iterator).

T	e	s	t	‿	Ξ	ε	σ	κ	ά	π	ά	ζ	ω	‿	ر	ح	ي	م	‿	ر	ح	ي	م	‿	e	x	t	e	́	n	d	↵	i	t	ş	ţ	ş	ţ	
0054	0065	0073	0074	0020	039E	03B5	03C3	03BA	03AC	03C0	03AC	03B6	03C9	0020	0631	062D	064A	0645	0020	0631	062D	064A	0645	0020	0065	0078	0074	0065	0301	006E	0064	000A	0069	0074	015F	0162	015F	0162	0000

Script: Arab

UAX #29: Unicode Text Segmentation

3.1.1. For Arabic runs additionally look for kashida justification points in each word.

```
Find the priority of the connecting opportunities in each word
Add expansion at the highest priority connection opportunity
If more than one connection opportunity have the same highest value, use the opportunity closest to the end of the word.
```

Priority	Glyph	Condition	Kashida Location
1	User inserted	The user entered a Kashida in a position.	After the user inserted kashida
2	Seen and Sad	Connecting to the next character.	After the character.
3	Taa Marbutah, Haa, Dal	Connecting to previous character.	Before the final form of these characters.
4	Alef, Tah Lam and Caf and Gaf	Connecting to previous character.	Before the final form of these characters.
5	RA, Ya and Alef Maqsurah	Connected to medial BAA	Before preceding medial BAA
6	WAW, Ain, Qaf and Fa	Connecting to previous character.	Before the final form of these characters.
7	Other connecting characters	Connecting to previous character.	Before the final form of these characters.

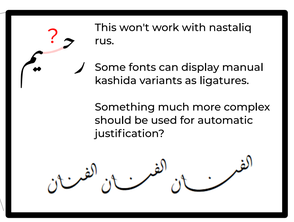
Overview of the other justification techniques

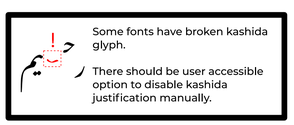
Justifying Text using Cascading Style Sheets (CSS) in Internet Explorer 5.5 (via Wayback Machine)

Shape single code point (U+0640) run into kashida cluster: ـ 40

3.2.1. If shaped successfully, insert as many kashida clusters as possible.

3	ٱ 1ed	ٰ 1fa	ـ 40	ـ 40	ـ 40	> 1c0	ّ 31	3
19...19	18...18	17...17				16...16	15...15	24...24





3.2.2. Elongate (change advance) or insert spaces in the rest of word bounds.

u bc0	ٱ b21	→	u bc0	3	ٱ b21
n...n	m...m		n...n		m...m

Output

Test Ξεσκεπάζω رحيم exténd itşţşţ

Generate justification opportunies — Justification opportunies search code

```
UBreakIterator *bi = ubrk_open(UBRK_WORD, lang, data, data.size, &err);
if (U_FAILURE(err)) {
    //No data - use fallback
    int64_t limit = 0;
    for (int64_t i = 0; i < data.size; i++) {
        if (u_isWhitespace(get_char(i))) {
            generate_kashida_justification_opportunies(limit, i, p_ops);

            op.position = i;
            op.kashida = false;

            limit = i + 1;
        }
    }
    generate_kashida_justification_opportunies(limit, data.size, p_ops);
    return;
}
int64_t limit = 0;
while (ubrk_next(bi) != UBRK_DONE) {
    if (ubrk_getRuleStatus(bi) != UBRK_WORD_NONE) {
        generate_kashida_justification_opportunies(limit, ubrk_current(bi), p_ops);

            op.position = ubrk_current(bi);
            op.kashida = false;

        limit = ubrk_current(bi) + 1;
    }
}
ubrk_close(bi);
```

Generate kashida justification opportunies — Kashida justification opportunies search code

```
int32_t kashida_pos = -1;
int8_t priority = 100;
int64_t i = p_start;

uint32_t pc = 0;

while ((p_end > p_start) && is_transparent(get_char(p_end - 1)))
    p_end--;

while (i < p_end) {
    uint32_t c = get_char(i);

    if (c == 0x0640) {
        kashida_pos = i;
        priority = 0;
    }
    if (priority >= 1 && i < p_end - 1) {
        if (is_seen_sad(c) && (get_char(i + 1) != 0x200C)) {
            kashida_pos = i;
            priority = 1;
        }
    }
    if (priority >= 2 && i > p_start) {
        if (is_teh_marbuta(c) || is_dal(c) || (is_heh(c) && i == p_end - 1)) {
            if (is_connected_to_prev(c, pc)) {
                kashida_pos = i - 1;
                priority = 2;
            }
        }
    }
    if (priority >= 3 && i > p_start) {
        if (is_alef(c) || ((is_lam(c) || is_tah(c) || is_kaf(c) || is_gaf(c)) && i == p_end - 1)) {
            if (is_connected_to_prev(c, pc)) {
                kashida_pos = i - 1;
                priority = 3;
            }
        }
    }
    if (priority >= 4 && i > p_start && i < p_end - 1) {
        if (is_beh(c)) {
            if (is_reh(get_char(i + 1)) || is_yeh(get_char(i + 1))) {
                if (is_connected_to_prev(c, pc)) {
                    kashida_pos = i - 1;
                    priority = 4;
                }
            }
        }
    }
    if (priority >= 5 && i > p_start) {
        if (is_waw(c) || ((is_ain(c) || is_qaf(c) || is_feh(c)) && i == p_end - 1)) {
            if (is_connected_to_prev(c, pc)) {
                kashida_pos = i - 1;
                priority = 5;
            }
        }
    }
    if (priority >= 6 && i > p_start) {
        if (is_reh(c)) {
            if (is_connected_to_prev(c, pc)) {
                kashida_pos = i - 1;
                priority = 6;
            }
        }
    }
    if (!is_transparent(c)) pc = c;
    i++;
}
if (kashida_pos > -1) {

            op.position = kashida_pos;
            op.kashida = true;

}
```

Line justification — Kashida insertion

```
//Step 1: Kashida justification
if ((p_flags == TEXT_JUSTIFICATION_KASHIDA_AND_WHITESPACE) || (p_flags == TEXT_JUSTIFICATION_KASHIDA_ONLY)) {
    Cluster ks_cluster;
    float ks_width_per_op = (p_width - width) / ks_count;
    for (int64_t i = 0; i < jst_ops.size(); i++) {
        if ((jst_ops[i].position <= 0) || jst_ops[i].position >= data_size - 1)
            continue;
        if (jst_ops[i].kashida) {
            int64_t j = i;
            while (j < visual.size()) {
                if (visual[j].start == jst_ops[i].position) {
                    _shape_single_cluster(visual[j - 1].end, visual[j].start, HB_DIRECTION_RTL, HB_SCRIPT_ARABIC, 0x0640, visual[j].font_face, ks_cluster);
                    ks_cluster.ignore_on_input = true;

                    //Add new kashda multiple times
                    if (ks_cluster.width > 0) {
                        int64_t ks_count_per_op = ks_width_per_op / ks_cluster.width;
                        for (int64_t k = 0; k < ks_count_per_op; k++) {
                            visual.insert(visual.begin() + j, ks_cluster);
                            j++;
                        }

                        width += ks_count_per_op * ks_cluster.width;
                    }
                }
                j++;
            }
        }
    }
}
```

Space insertion

```
//Step 2: Whitespace justification
if ((p_flags == TEXT_JUSTIFICATION_KASHIDA_AND_WHITESPACE) || (p_flags == TEXT_JUSTIFICATION_WHITESPACE_ONLY)) {
    float ws_width_per_op = (p_width - width) / ws_count;

    if (base_font.is_null()) {
        return width;
    }
    Ref<TLFontFace> _font = base_font->get_face(base_style);
    if (_font.is_null()) {
        return width;
    }

    Cluster ws_cluster;
    ws_cluster.font_face = _font.ptr();
    ws_cluster.cl_type = _CLUSTER_TYPE_SKIP;
    ws_cluster.valid = true;
    ws_cluster.ascent = _font->get_ascent(base_size);
    ws_cluster.descent = _font->get_descent(base_size);
    ws_cluster.width = ws_width_per_op;
    ws_cluster.glyphs.push_back(Glyph(0, Point2(), Point2(ws_width_per_op, 0)));
    ws_cluster.ignore_on_input = true;

    for (int64_t i = 0; i < jst_ops.size(); i++) {
        if ((jst_ops[i].position <= 0) || jst_ops[i].position >= data_size - 1)
            continue;
        if (!jst_ops[i].kashida) {
            int64_t j = 0;
            while (j < visual.size()) {
                if (visual[j].start == jst_ops[i].position) {
                    if ((visual[j].glyphs.size() == 1) && u_isWhitespace(get_char(visual[j].start))) {
                        //Extend existing whitespace
                        visual[j].glyphs[0].advance.x += ws_width_per_op;
                        visual[j].width += ws_width_per_op;
                    } else {
                        //Add new whitespace
                        ws_cluster.is_rtl = visual[j].is_rtl;
                        if (visual[j].is_rtl) {
                            ws_cluster.start = visual[j].end + 1;
                            ws_cluster.end = visual[j].end + 1;

                            j++;
                            visual.insert(visual.begin() + j, ws_cluster);
                            j++;
                        } else {
                            ws_cluster.start = visual[j].start;
                            ws_cluster.end = visual[j].start;

                            visual.insert(visual.begin() + j, ws_cluster);
                            j++;
                        }
                    }
                    width += ws_width_per_op;
                }
                j++;
            }
        }
    }
}
```

gdtl module source code links

general documentation/info links

CC 0